

JEONME

Dokumentasi Docker & Infrastruktur

Dockerfile, Docker Compose, Nginx & TLS — Local, Staging, Production

Dokumen	INFRA-DOC v1.0
Status	Berfungsi, sudah diaudit
Tanggal	3 Juli 2026
Stack	Docker + Docker Compose · Nginx · Let's Encrypt (Certbot)
Lokasi kode	<code>docker/</code> , <code>docker-compose*.yaml</code>

1. Prinsip Strategi Container

Image Docker dibangun **sekali** di CI (job `build-images` pada `ci.yml`), lalu image yang **sama persis** dipakai di staging maupun production — bukan build ulang di server. Ini menjamin apa yang diuji di CI adalah apa yang sungguhan berjalan di production (lihat CICD-GUIDE Bagian 1).

2. Dockerfile API (`docker/api/Dockerfile`)

Multi-stage: `builder` (`golang:1.25-alpine`) mengompilasi binary statis (`CGO_ENABLED=0`), lalu `runner` (`alpine:3.20`) hanya menyalin binary jadi + folder `migrations` . Hasil akhir: image sangat kecil (~15–20MB) tanpa toolchain Go sama sekali di image final.

STAGE	BASE IMAGE	ISI
<code>builder</code>	<code>golang:1.25-alpine</code>	<code>go mod download</code> → <code>go build -ldflags="-w -s"</code> (strip simbol debug, image lebih kecil)
<code>runner</code>	<code>alpine:3.20</code>	Binary <code>api</code> + folder <code>migrations</code> (dipakai subcommand <code>migrate</code> saat deploy) + <code>ca-certificates</code> , <code>tzdata</code>

Expose port `8080` , entryptoint langsung `./api` (tanpa argumen = jalankan server; dengan argumen `migrate ...` = jalankan migrasi lalu keluar, lihat Dokumentasi Backend Bagian 8).

3. Dockerfile Web (`docker/web/Dockerfile`)

Tiga stage: `deps` (install dependency), `builder` (`next build` , menghasilkan `.next/standalone`), `runner` (hanya menyalin hasil standalone + static + public — **tanpa** `node_modules` penuh).

STAGE	BASE IMAGE	ISI
<code>deps</code>	<code>node:24-alpine</code>	<code>npm ci</code> (instal reproducible dari <code>package-lock.json</code>)
<code>builder</code>	<code>node:24-alpine</code>	<code>next build</code> — butuh <code>output: 'standalone'</code> di <code>next.config.js</code>
<code>runner</code>	<code>node:24-alpine</code>	<code>public/</code> + <code>.next/standalone</code> + <code>.next/static</code> , CMD <code>["node", "server.js"]</code>

Node dinaikkan dari versi 20 ke 24 (Active LTS) di kedua Dockerfile — Node 20 sudah EOL per 30 April 2026 (lihat README).

4. Docker Compose — Local Dev (`docker-compose.yml`)

SERVICE	IMAGE/BUILD	PORT	CATATAN
---------	-------------	------	---------

web	build dari docker/web/Dockerfile	3000	NEXT_PUBLIC_API_BASE_URL=http://localhost:8080/api/v1
api	build dari docker/api/Dockerfile	8080	depends_on : db (healthy), redis (started)
db	postgres:16-alpine	5432	Healthcheck pg_isready ; migrasi tidak auto-run — jalankan manual make migrate-up
redis	redis:7-alpine	6379	—

5. Docker Compose — Staging & Production (VPS Shared)

Perubahan arsitektur: deployment nyata memakai VPS **shared** yang sudah menjalankan puluhan situs klien lain lewat Apache. Karena itu `docker-compose.staging.yml` dan `.prod.yml` **tidak lagi punya service nginx/certbot** — port 80/443 host sudah dipakai Apache untuk domain lain. Container `web` / `api` di-bind hanya ke `127.0.0.1`, lalu Apache di level sistem yang mem-proxy domain publik ke port tersebut. Lihat Dokumentasi CI/CD & Deployment Bagian 2 untuk arsitektur lengkap.

SERVICE	STAGING	PRODUCTION
web	Image GHCR, port host <code>127.0.0.1:\${'{'}}WEB_HOST_PORT{'}'}</code> :3000 (staging: 23100, production: 23000)	
api	Image GHCR, port host <code>127.0.0.1:\${'{'}}API_HOST_PORT{'}'}</code> :8080 (staging: 28180, production: 28080). <code>depends_on</code> : db (condition <code>service_healthy</code>), redis (started)	
db	postgres:16-alpine	postgres:16-alpine
redis	redis:7-alpine	
worker (dikomentari)	TODO: <code>main.go</code> belum punya subcommand <code>worker</code> — perlu job queue (mis. <code>asynq</code>) sebelum diaktifkan	

Bug rollout yang diperbaiki: db awalnya tidak punya healthcheck, sehingga `docker compose run --rm api migrate up` bisa mulai sebelum Postgres benar-benar siap menerima koneksi (race condition, gejala: "connection refused"). Sekarang db punya healthcheck `pg_isready` dan api memakai `depends_on`: `condition: service_healthy`.

6. Reverse Proxy & TLS — Apache (Bukan Nginx-in-Docker)

Karena VPS shared, TLS & reverse proxy ditangani **Apache sistem** (sudah terpasang untuk semua situs lain di server), bukan container Nginx. Folder `docker/nginx/` di repo **masih ada** sebagai referensi kalau nanti pindah ke VPS terdedikasi, tapi **tidak dipakai** di deployment saat ini.

DOMAIN	VHOST APACHE	PROXY TARGET
jeonme.com, www.jeonme.com	/etc/apache2/sites-available/jeonme.com.conf	/api/ → 127.0.0.1:28080, / → 127.0.0.1:23000

staging.jeonme.com	/etc/apache2/sites-available/staging.jeonme.com.conf	/api/ → 127.0.0.1:28180, / → 127.0.0.1:23100
--------------------	--	--

Sertifikat diterbitkan lewat `certbot certonly --webroot` (bukan plugin `--apache` otomatis, supaya vhost tidak diedit otomatis oleh Certbot). Renewal tercakup `certbot.timer` sistem yang sudah berjalan 2x sehari untuk semua sertifikat di server ini — **tidak perlu** cron tambahan khusus Jeonme.

Detail vhost lengkap & katalog bug rollout ada di Dokumentasi CI/CD & Deployment Bagian 2 & 10.1.

8. Environment Variables — Rujukan Lengkap

FILE	DIPAKAI OLEH	VARIABEL KUNCI
<code>.env.example</code> (root)	VPS staging/production, dibaca <code>docker compose</code>	<code>GHCR_REPO</code> , <code>IMAGE_TAG</code> , <code>DATABASE_URL</code> , <code>POSTGRES_PASSWORD</code> , <code>JWT_SECRET</code> , <code>XENDIT_*</code> , <code>CORS_ALLOWED_ORIGINS</code> , <code>NEXT_PUBLIC_API_BASE_URL</code>
<code>apps/api/.env.example</code>	Development backend lokal (tanpa Docker)	<code>APP_ENV</code> , <code>APP_PORT</code> , <code>DATABASE_URL</code> , <code>REDIS_URL</code> , <code>JWT_SECRET</code> , <code>XENDIT_*</code> , <code>CORS_ALLOWED_ORIGINS</code>
<code>apps/web/.env.example</code>	Development frontend lokal	<code>NEXT_PUBLIC_API_BASE_URL</code>

Ganti seluruh nilai `CHANGE_ME` sebelum production, dan jangan pernah commit file `.env` asli (sudah masuk `.gitignore`).

9. Volumes & Persistensi Data

VOLUME	DIPAKAI DI	ISI
<code>pgdata</code>	local dev & production	Data PostgreSQL
<code>pgdata_staging</code>	staging	Data PostgreSQL staging (terpisah dari production)

Sertifikat TLS (`/etc/letsencrypt/`) dan webroot ACME challenge (`/var/www/certbot/`) dikelola di level **sistem VPS**, dipakai bersama oleh Apache untuk semua situs di server — bukan volume Docker milik Jeonme.

10. Catatan Versi & Keamanan

Ringkasan dari README — perubahan versi yang ditemukan & diperbaiki saat penyusunan boilerplate:

- **Next.js 14.2.5 → 16.2.10 LTS** — versi 14 sudah *end-of-life* sejak Oktober 2025, rentan CVE kritis App Router (RCE "React2Shell", CVE-2025-66478). React ikut naik ke 19.2.7.
- **Node.js 20 → 24 (Active LTS)** di Dockerfile & CI — Node 20 EOL 30 April 2026.
- **Go 1.22 → 1.25** di `go.mod`, Dockerfile, CI — Go hanya mendukung dua rilis mayor terbaru.
- Masih ada 1 kerentanan moderate (XSS PostCSS) dari dependency internal Next.js sendiri, bukan dependency langsung proyek — pantau lewat `npm audit` (sudah otomatis berjalan non-blocking di CI, lihat Dokumentasi

CI/CD).